

Reviewer Pack — Euler Characteristic of Matrix Factorization and Resolution ...

Entry e1ba315c571d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 15:26:53 UTC

Euler Characteristic of Matrix Factorization and Resolution Tree Width

Entry ID: e1ba315c571d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 15:26:53 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry: Matrix Factorization Theory **Field B** (complexity object): Complexity Theory: Resolution Tree Width

Statement:

[‘For every Tseitin formula G , the resolution tree width $L(G)$ satisfies $L(G) \geq 2^{(2\nu(G))}$ where $\nu(G)$ is the Euler characteristic of the matrix factorization associated with G .’, ‘This implies that for a Tseitin formula with a non-zero Euler characteristic, its resolution tree width is super-polynomial in the size of the formula.’, ‘The conjecture holds if and only if there exists an instance of the Tseitin formula with an Euler characteristic greater than zero.’]

Rationale (proposer’s reasoning):

[‘Matrix factorization theory provides a bridge between algebraic geometry and complexity theory, potentially uncovering new structures in the resolution tree that are not apparent through traditional methods.’, ‘The Euler characteristic has been used in other complexity-theoretic contexts (e.g., in topological quantum computation) to derive lower bounds on complexity measures.’, ‘A positive correlation between the Euler characteristic of matrix factorization and resolution tree width could suggest a novel approach to proving super-polynomial lower bounds for Resolution.’]

Taxonomy category: AVG_T0_WORST_CASE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 73655b9a07b210d9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The Euler characteristic $\nu(G)$ of the matrix factorization associated with a Tseitin formula G , when compared to its resolution tree width $L(G)$, satisfies $L(G) \geq 2^{(2\nu(G))}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Euler characteristic" AND "matrix factorization" AND "resolution tree width" - "Tseitin formula" AND "resolution tree width" AND "Euler characteristic" - "super-polynomial complexity" AND "matrix factorization" AND Tseitin

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i + 1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            factor = Fraction(A[i][i])
            for j in range(n):
                A[i][j] /= factor
            for j in range(m):
                if j != i:
                    factor = Fraction(A[j][i])
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def resolution_tree_width(G):
        # Placeholder function to compute resolution tree width
        # This is a stub and should be replaced with actual computation
        return random.randint(1, 10)
```

```

def euler_characteristic(A):
    m, n = len(A), len(A[0])
    det = Fraction(1)
    for i in range(m):
        det *= A[i][i]
    return det

def tseitin_formula(n):
    # Placeholder function to generate a Tseitin formula
    # This is a stub and should be replaced with actual generation
    return [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

n = random.choice([5, 10, 15, 20, 30, 40])
G = tseitin_formula(n)
A = gaussian_elimination(G)
v_G = euler_characteristic(A)
L_G = resolution_tree_width(G)

if v_G == 0:
    return {
        "metric_name": "Resolution Tree Width",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "Euler characteristic is zero"
    }

metric_value = L_G >= 2**(2 * v_G)
return {
    "metric_name": "Resolution Tree Width",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": metric_value,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    total_metric_value = 0

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        if trial_result["metric_value"] is not None:
            total_metric_value += trial_result["metric_value"]
        results.append(trial_result)

    mean_metric_value = total_metric_value / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Euler characteristic is zero\" first_failing_seed={first_failing_seed}")

```

```

else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'Resolution Tree Width', 'metric_value': True, 'instances_tested': 1, 'conjecture': 'The conjecture is true'}
TRIAL: {'metric_name': 'Resolution Tree Width', 'metric_value': True, 'instances_tested': 1, 'conjecture': 'The conjecture is true'}
TRIAL: {'metric_name': 'Resolution Tree Width', 'metric_value': True, 'instances_tested': 1, 'conjecture': 'The conjecture is true'}
TRIAL: {'metric_name': 'Resolution Tree Width', 'metric_value': True, 'instances_tested': 1, 'conjecture': 'The conjecture is true'}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_45d223fb.py", line 86, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_45d223fb.py", line 58, in run_trial
    A = gaussian_elimination(G)
        ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_45d223fb.py", line 31, in gaussian_elimination
    A[i][j] /= factor
  File "/usr/lib/python3.12/fractions.py", line 615, in forward
    return monomorphic_operator(a, b)
           ^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 763, in _div
    raise ZeroDivisionError('Fraction(%s, 0)' % db)
ZeroDivisionError: Fraction(1, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error, which prevented the production of data necessary to evaluate the conjecture. | next: Investigate and fix the crash in the gaussian_elimination function to allow for proper testing of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15269
2	preregistration	ollama_remote	glm4:latest	0	0	9699
3	novelty	ollama_remote	glm4:latest	0	0	8368

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	8582
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12200
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12874
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16023
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9779
9	judge	ollama_remote	glm4:latest	0	0	26573

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 119368 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e1ba315c571d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e1ba315c571d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e1ba315c571d.tar.gz` (if generated)